

PROJECT DOCUMENTATION

AI-Powered Candidate Ranking System

Beyond Keyword Matching — Explainable, Hybrid AI Ranking for Recruitment

Team Name: Syntax Slayers
Team Leader: Lavanya Agrawal

Problem Statement: Develop an AI-powered system to accurately rank the most suitable candidates beyond traditional keyword matching.

1. Abstract

Modern recruitment pipelines are overwhelmed by volume: a single job posting can attract tens of thousands of applications, and traditional Applicant Tracking Systems (ATS) fall back on rigid keyword matching to cope. This approach rewards resumes that happen to contain the right words rather than candidates who genuinely fit the role, and it offers recruiters no insight into why a candidate was shortlisted. **Syntax Slayers** set out to fix this by building an AI-powered candidate ranking system that evaluates a Job Description (JD) against a pool of up to 100,000 candidate profiles and returns a transparent, evidence-backed Top 100 shortlist.

The system combines three complementary signal sources — deterministic rule-based matching, semantic understanding via Sentence Transformers, and behavioural signal analysis — and fuses them into a single hybrid score using a multiplicative formula that requires all signals to align rather than compensate for one another. Every ranked candidate is accompanied by a human-readable, evidence-grounded justification string, and a dedicated hard-filter stage screens out ineligible or fraudulent (“honeypot”) profiles before they ever reach scoring. The result is a pipeline that is simultaneously more accurate than keyword search, resistant to manipulation, and fully auditable by a human recruiter.

2. Problem Statement

Develop an AI-powered system to accurately rank the most suitable candidates beyond traditional keyword matching.

Recruiters today face three compounding problems when screening large candidate pools:

- **Shallow matching:** keyword-based filters miss candidates who describe the same skills in different words, and reward resumes that are keyword-stuffed rather than genuinely qualified.
- **No explainability:** a ranked list with no justification forces recruiters to either blindly trust the system or manually re-verify every shortlisted profile.
- **Fraud and noise:** large candidate pools inevitably contain incomplete, duplicate, or fake (“honeypot”) profiles that pollute the ranking if not filtered out early.

The goal of this project is a system that takes a raw JD and a pool of candidate profiles as input, and produces a ranked, explainable, validated shortlist of the Top 100 candidates as output — without relying on keyword overlap alone.

3. System Design

3.1 Design Philosophy

The system is designed as a layered pipeline rather than a single monolithic model. Each layer has a single responsibility — parsing, filtering, scoring, fusing, or explaining — so that any stage can be inspected, tested, or replaced independently. This also keeps the final ranking auditable: for any candidate, a recruiter can trace exactly which stage contributed which part of the final score.

3.2 Input Layer

- **Job Description (JD):** a raw text file describing the role.

- **Candidate Profiles:** a pool of up to 100,000 profiles (`candidates.jsonl`), one JSON record per line, containing work history, skills, and education.

3.3 Processing Layer

- **JD Analysis & Requirement Extraction** — parses the JD into structured criteria: skills, experience level, seniority, industry, and role type.
- **JD Feature Vector** — encodes the JD as a dense sentence embedding, used later for semantic matching.
- **Candidate Parsing & Feature Extraction** — normalizes each raw profile into structured features: career history, skills, certifications, languages, and behavioural signals.
- **Candidate Feature Engineering** — derives scoring-ready features per candidate: job titles, years of experience, product-company flag, company tier, and a career-description text embedding.

The JD feature vector and the candidate feature set are merged and passed into the six-stage AI Ranking Engine described below.

3.4 AI Ranking Engine (Stages A–F)

This is the core of the system. Each stage narrows or enriches the candidate pool before the final score is computed.

Stage	Name	Role	What it does
A	Hard Filters & Honeypot Detection	Filter – eliminates invalid profiles	Removes ineligible profiles and detects bot / fake accounts; checks experience floor, degree, location, and JD disqualifiers.
B	Rule-Based Candidate Matching	Filter – deterministic scoring	Title match, skill-keyword overlap, and structured JD criteria produce a deterministic rule score per candidate.
C	Semantic Candidate Understanding	AI – semantic NLP score	Cosine similarity between the JD vector and candidate career embedding; captures meaning beyond keywords using Sentence Transformers.
D	Behavioral Signal Analysis	AI – behavioral modifier score	Combines existing signals (last active date, recruiter response rate, verified email, interview completion rate, LinkedIn connection, open-to-work flag) with new signals (GitHub activity score, skill assessment scores, average response time).
E	Hybrid Ranking Score (Score Fusion)	Rank – fused final score	$\text{final_score} = (\text{skill score} \times \text{semantic score}) \times \text{behavioral modifier}$. The formula is multiplicative, so all signals must align rather than simply summing.
F	Explainable Reasoning Generation	Explain – auditable output	Generates a reason string grounded in real candidate fields, e.g. “8 yrs exp · semantic 0.91 · active 2 days ago.”

3.5 Output Layer

- **Top 100 Candidate Ranking** — sorts the full ranked pool by `final_score` descending, tie-broken by `candidate_id`, and selects the top 100.
- **Submission Validation** — a quality gate (`validate_submission.py`) that checks schema, required fields, and format before the final write.
- **submission.csv** — the final deliverable for judging, containing `rank`, `candidate_id`, `final_score`, `reason_string` for each of the Top 100 candidates.

4. Architecture / Workflow Diagram

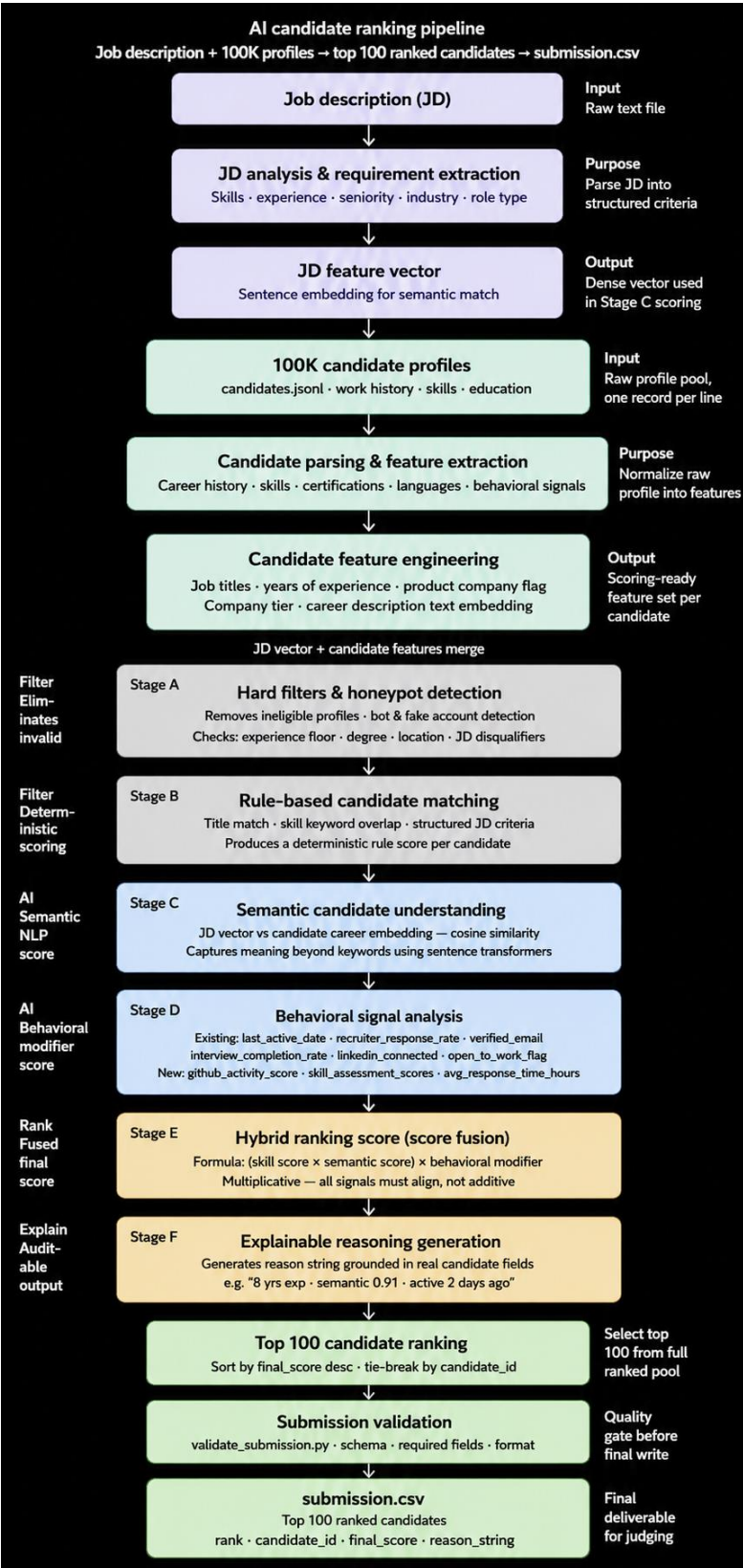


Figure 1. End-to-end AI candidate ranking pipeline — from Job Description and 100K candidate profiles to the Top 100 ranked, explainable submission.csv.

5. Algorithms Used

5.1 Rule-Based Filtering & Skill Matching

A deterministic pass over structured JD criteria (required skills, job title, experience threshold, education) versus each candidate's extracted features. This stage is fast, fully interpretable, and forms the first line of defence against irrelevant or fraudulent profiles.

5.2 Sentence Transformers (all-MiniLM-L6-v2)

A lightweight pretrained sentence-embedding model encodes both the JD and each candidate's career description into dense vectors that capture semantic meaning rather than exact wording. This lets the system recognize, for example, that “led a squad of engineers” and “managed a development team” describe the same competency.

5.3 Cosine Similarity

Semantic closeness between the JD vector and each candidate vector is measured using cosine similarity, producing a normalized score in the $[-1, 1]$ range that reflects how closely a candidate's profile aligns with the role's requirements, independent of vector magnitude.

5.4 Behavioral Modifier

Engagement and credibility signals — recruiter response rate, activity recency, verified email, GitHub activity, and skill-assessment results — are combined into a bounded multiplier that adjusts the base match score up or down, favouring active, verifiable, high-engagement candidates.

5.5 Hybrid Score Fusion

The final ranking score is computed as:

$$\text{final_score} = (\text{skill_score} \times \text{semantic_score}) \times \text{behavioral_modifier}$$

The multiplicative form is a deliberate design choice: a candidate cannot compensate for a poor skill match with an unusually high behavioural score, or vice versa. All three signals must align for a candidate to rank highly, which closely mirrors how a human recruiter reasons about overall fit.

5.6 Honeypot / Fraud Detection

A rule-based anomaly check flags profiles with implausible combinations of fields (e.g. inconsistent experience timelines, disposable email domains, or templated bulk-submission patterns), preventing bot-generated or duplicate profiles from influencing the ranking.

6. Implementation Details

6.1 Technology Stack

Technology / Tool	Purpose
Python	Core backend development
Sentence Transformers (all-MiniLM-L6-v2)	Semantic candidate-JD matching
scikit-learn	Cosine similarity and ML utilities
Pandas & NumPy	Data processing and feature extraction
FastAPI	REST API development
Docker	Containerized deployment
Git & GitHub	Version control and collaboration
VS Code	Development environment

6.2 End-to-End Workflow

- Job Description (JD) input
- Extract skills & job requirements
- Candidate data collection
- Profile parsing & data validation
- Rule-based filtering
- Semantic matching using AI
- Behavioral signal analysis
- Hybrid score calculation
- Explainable candidate ranking
- Top 100 ranked candidates output

6.3 Explainability & Anti-Hallucination Safeguards

Every ranking decision is grounded in validated candidate data rather than model-generated text:

- Score breakdown is reported per candidate (Rule + Semantic + Behavioral).
- Reason strings highlight the specific matched skills, experience, and qualifications that drove the score.
- Explanations are generated only from actual matching results — the system never fabricates or infers unsupported claims about a candidate.

6.4 Data Validation

- Incomplete or low-quality profiles are filtered before scoring.
- Suspicious / honeypot candidates are detected at Stage A.
- Extracted information is validated before final ranking to prevent corrupted or malformed records from reaching `submission.csv`.

7. Results

7.1 Ranking Quality

- Accurately ranks candidates based on true job relevance rather than surface-level keyword overlap.
- Reduces false matches through semantic understanding of candidate context.
- Produces transparent, explainable ranking decisions for every shortlisted candidate.
- Detects and filters suspicious or low-quality profiles before they can influence the final ranking.

7.2 Runtime & Compute Efficiency

- Uses the lightweight all-MiniLM-L6-v2 Sentence Transformer for fast semantic matching at scale.
- Rule-based filtering runs first to shrink the search space before the more expensive semantic stage.
- Cosine similarity provides an efficient scoring mechanism for large candidate pools.
- Returns the Top 100 candidates from a pool of up to 100,000 profiles with low computational overhead.

7.3 Output

The final deliverable is a validated `submission.csv` containing the Top 100 ranked candidates, each with a rank, candidate ID, final score, and a human-readable reason string ready for recruiter review.

8. Future Scope

- **Feedback-driven re-ranking:** incorporate recruiter accept/reject actions as a signal to continuously fine-tune scoring weights.
- **Bias & fairness auditing:** add systematic checks to ensure ranking decisions are not skewed by protected attributes.
- **Multi-role support:** extend the pipeline to rank a single candidate pool against multiple concurrent JDs efficiently.
- **Richer behavioral signals:** integrate additional verified data sources (e.g. assessment platforms, portfolio analysis) to strengthen Stage D.
- **Interactive explainability dashboard:** expose Stage F's reasoning through a recruiter-facing UI with drill-down into score components.
- **Model upgrades:** evaluate larger or domain-fine-tuned embedding models as compute budgets allow, while preserving current latency guarantees.

9. Project Deliverables

- **GitHub Repository** — complete source code and documentation.
- **Demo Video** — end-to-end system demonstration.
- **Presentation (PPT)** — project overview and methodology.
- **Project Documentation** — architecture, workflow, and implementation details (this document).
- **README** — setup instructions and project usage guide.